Screenshot 1 – PostgreSQL Database Creation

```
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % cd ~/Tool-setup/lab06
python create_database.py
Checking AWS network...
Creating database security group...
Created Lab 06 database security group.
Creating database subnet group...
Created Lab 06 database subnet group.
Checking database password in SSM...
New database password saved in SSM.
Creating private PostgreSQL database...
Database creation started.
Database ID: itmo-444-544-weather-db
The database may take several minutes to become available.
Password is stored in SSM and is not displayed.
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```

Created the PostgreSQL database, security group, subnet group, and saved the password in AWS SSM.

Screenshot 2 – PostgreSQL Database Available

```
  --region us-east-2 \
  --query 'DBInstances[0].[DBInstanceIdentifier,DBInstanceStatus]' \
  --output table
----------------------------
|    DescribeDBInstances    |
+---------------------------+
|  itmo-444-544-weather-db  |
|  available                |
+---------------------------+
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```

Verified that the PostgreSQL database is running and available in AWS RDS.

Screenshot 3 – Weather Auto Scaling Group Creation

```
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % aws autoscaling create-auto-scaling-group \

  --auto-scaling-group-name itmo-444-544-weather-asg \
  --launch-template LaunchTemplateName=itmo-444-544-weather-lt,Version='$Latest' \
  --min-size 1 \
  --max-size 2 \
  --desired-capacity 1 \
  --vpc-zone-identifier "subnet-0279c7d7ebf196008,subnet-0eb4fa6c23df7087c" \
  --target-group-arns arn:aws:elasticloadbalancing:us-east-2:100995727963:targetgroup/itmo-444
-544-weather-tg/78faf130642679d8 \
  --health-check-type ELB \
  --health-check-grace-period 300 \
  --region us-east-2
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```

Screenshot 4 – Weather Application Health Check

```
------------------------------
|  DescribeAutoScalingGroups |
+----------------------------+
|  itmo-444-544-weather-asg  |
|  1                         |
|  i-01e80e3c62017df35       |
|  InService                 |
|  Healthy                   |
+----------------------------+
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % curl -i --max-time 15 \
  http://itmo-444-544-weather-alb-116193645.us-east-2.elb.amazonaws.com:8080/health
HTTP/1.1 200 OK
Date: Thu, 05 Nov 2026 00:41:26 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 69
Connection: keep-alive
X-Powered-By: Express
ETag: W/"45-avKcdEWbcbfijjJquYufrriN3vQ"

{"status":"ok","service":"weather","time":"2026-11-05T00:41:26.600Z"}%
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```

Screenshot 5 – PostgreSQL Weather History

```
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % curl -i --max-time 30 \
  http://itmo-444-544-weather-alb-116193645.us-east-2.elb.amazonaws.com:8080/history
HTTP/1.1 200 OK
Date: Thu, 05 Nov 2026 00:43:33 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 2242
Connection: keep-alive
X-Powered-By: Express
ETag: W/"8c2-Cfs43PtBdsOfhab7bWzjoc1hJbM"

{"history":[{"zip":"60616","data":{"zip":"60616","city":"Chicago","state":"Illinois","forecast
":[{"name":"Tonight","windSpeed":"15 mph","temperature":50,"shortForecast":"Areas Of Drizzle",
"windDirection":"SW","temperatureUnit":"F","detailedForecast":"Areas of drizzle. Cloudy, with
a low around 50. Southwest wind around 15 mph, with gusts as high as 25 mph. Chance of precipi
tation is 40%."},{"name":"Thursday","windSpeed":"10 to 15 mph","temperature":53,"shortForecast
":"Chance Rain Showers","windDirection":"WSW","temperatureUnit":"F","detailedForecast":"A chan
ce of rain showers. Cloudy. High near 53, with temperatures falling to around 51 in the aftern
oon. West southwest wind 10 to 15 mph, with gusts as high as 25 mph. Chance of precipitation i
s 50%. New rainfall amounts less than a tenth of an inch possible."},{"name":"Thursday Night",
"windSpeed":"10 to 15 mph","temperature":43,"shortForecast":"Rain Likely then Cloudy","windDir
ection":"NNW","temperatureUnit":"F","detailedForecast":"Rain likely before 11pm. Cloudy, with
a low around 43. North northwest wind 10 to 15 mph. Chance of precipitation is 70%. New rainfa
ll amounts less than a tenth of an inch possible."},{"name":"Friday","windSpeed":"15 mph","tem
perature":50,"shortForecast":"Slight Chance Rain","windDirection":"NNW","temperatureUnit":"F",
"detailedForecast":"A slight chance of rain. Partly sunny, with a high near 50. North northwes
t wind around 15 mph, with gusts as high as 25 mph. Chance of precipitation is 20%."},{"name":
"Friday Night","windSpeed":"10 to 15 mph","temperature":40,"shortForecast":"Partly Cloudy","wi
ndDirection":"NNW","temperatureUnit":"F","detailedForecast":"Partly cloudy, with a low around
40. North northwest wind 10 to 15 mph."},{"name":"Saturday","windSpeed":"5 to 10 mph","tempera
ture":50,"shortForecast":"Mostly Sunny","windDirection":"NW","temperatureUnit":"F","detailedFo
recast":"Mostly sunny, with a high near 50."},{"name":"Saturday Night","windSpeed":"5 mph","te
mperature":38,"shortForecast":"Mostly Clear","windDirection":"W","temperatureUnit":"F","detail
edForecast":"Mostly clear, with a low around 38."}],"latitude":41.8426,"longitude":-87.6306,"u
```

Screenshot 6: Advisor Auto Scaling Group Creation

```
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % aws autoscaling create-auto-scaling-group \

  --auto-scaling-group-name itmo-444-544-advisor-asg \
  --launch-template LaunchTemplateName=itmo-444-544-advisor-lt,Version='$Latest' \
  --min-size 1 \
  --max-size 2 \
  --desired-capacity 1 \
  --vpc-zone-identifier "subnet-0279c7d7ebf196008,subnet-0eb4fa6c23df7087c" \
  --target-group-arns arn:aws:elasticloadbalancing:us-east-2:100995727963:targetgroup/itmo-444
-544-advisor-tg/01866d7be405a55c \
  --health-check-type ELB \
  --health-check-grace-period 300 \
  --region us-east-2
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % aws autoscaling describe-auto-scaling-group
s \
  --auto-scaling-group-names itmo-444-544-advisor-asg \
  --region us-east-2 \
  --query 'AutoScalingGroups[*].Instances[*].[InstanceId,LifecycleState,HealthStatus]' \
  --output table
-------------------------------------------------
|           DescribeAutoScalingGroups           |
+---------------------+-------------+-----------+
|  i-029331c0077dafb6d |  InService  |  Healthy |
+---------------------+-------------+-----------+
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```

Screenshot 7: Advisor Application Health Check.

```
+---------------------+-------------+-----------+
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % curl -i --max-time 15 \
  http://itmo-444-544-weather-alb-116193645.us-east-2.elb.amazonaws.com:8081/health
HTTP/1.1 200 OK
Date: Thu, 05 Nov 2026 01:02:12 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 35
Connection: keep-alive
X-Powered-By: Express
ETag: W/"23-9aurPunHUwKjZ7dZGUp3yTiCP2M"

{"status":"ok","service":"advisor"}%
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```

Screenshot 8: Gemini AI Weather Advice.

```
        },
        {
            "name": "Saturday Night",
            "temperature": 38,
            "temperatureUnit": "F",
            "windSpeed": "5 mph",
            "windDirection": "W",
            "shortForecast": "Mostly Clear",
            "detailedForecast": "Mostly clear, with a low around 38."
        }
    ],
    "updatedAt": "2026-11-05T01:39:21.472Z",
    "source": "gemini",
    "advice": "Wear a jacket for the 50 degree temperature and wind. Also bring rain gear since there is drizzle.",
    "model": "gemini-3.5-flash-lite"
}
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```

Screenshot 9 — Unit Tests Passed

```
print("Weather URL saved.")
print("Advisor URL saved.")
PY
Weather URL saved.
Advisor URL saved.
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % python -m pytest tests/unit -q
....................                                                     [100%]
21 passed in 2.58s
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```

Take Screenshot 10 — Integration Tests Passed

```
==================================== test session starts ====================================
platform darwin -- Python 3.13.1, pytest-9.2.0, pluggy-1.7.0 -- /Users/hossamshribati/Tool-set
up/lab06/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/hossamshribati/Tool-setup/lab06
configfile: pytest.ini
collected 16 items

tests/integration/test_aws_resources.py::test_load_balancer_exists PASSED              [  6%]
tests/integration/test_aws_resources.py::test_target_groups_exist PASSED               [ 12%]
tests/integration/test_aws_resources.py::test_auto_scaling_group_exists[weather] PASSED [ 18%]
tests/integration/test_aws_resources.py::test_auto_scaling_group_exists[advisor] PASSED [ 25%]
tests/integration/test_aws_resources.py::test_iam_role_exists[weather] PASSED          [ 31%]
tests/integration/test_aws_resources.py::test_iam_role_exists[advisor] PASSED          [ 37%]
tests/integration/test_aws_resources.py::test_launch_template_exists[weather] PASSED   [ 43%]
tests/integration/test_aws_resources.py::test_launch_template_exists[advisor] PASSED   [ 50%]
tests/integration/test_aws_resources.py::test_postgresql_database_exists PASSED        [ 56%]
tests/integration/test_aws_resources.py::test_database_password_parameter_exists PASSED [ 62%]
tests/integration/test_stack_outputs.py::test_weather_url_exists PASSED                [ 68%]
tests/integration/test_stack_outputs.py::test_advisor_url_exists PASSED                [ 75%]
tests/integration/test_stack_outputs.py::test_alb_arn_exists PASSED                    [ 81%]
tests/integration/test_stack_outputs.py::test_alb_dns_exists PASSED                    [ 87%]
tests/integration/test_stack_outputs.py::test_weather_target_group_exists PASSED       [ 93%]
tests/integration/test_stack_outputs.py::test_advisor_target_group_exists PASSED       [100%]

==================================== 16 passed in 5.27s =====================================
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```

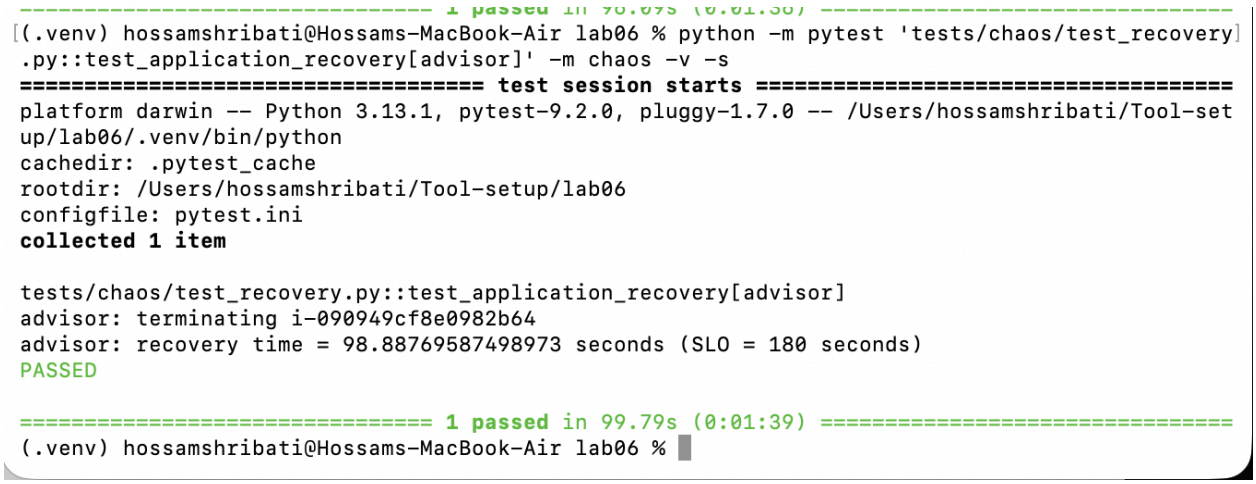
Screenshot 11 — Weather Chaos Test.

```
==================================== test session starts ====================================
platform darwin -- Python 3.13.1, pytest-9.2.0, pluggy-1.7.0 -- /Users/hossamshribati/Tool-set
up/lab06/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/hossamshribati/Tool-setup/lab06
configfile: pytest.ini
collected 1 item

tests/chaos/test_recovery.py::test_application_recovery[weather]
weather: terminating i-01e80e3c62017df35
weather: recovery time = 95.13729179199436 seconds (SLO = 180 seconds)
PASSED

================================== 1 passed in 96.09s (0:01:36) =================================
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```

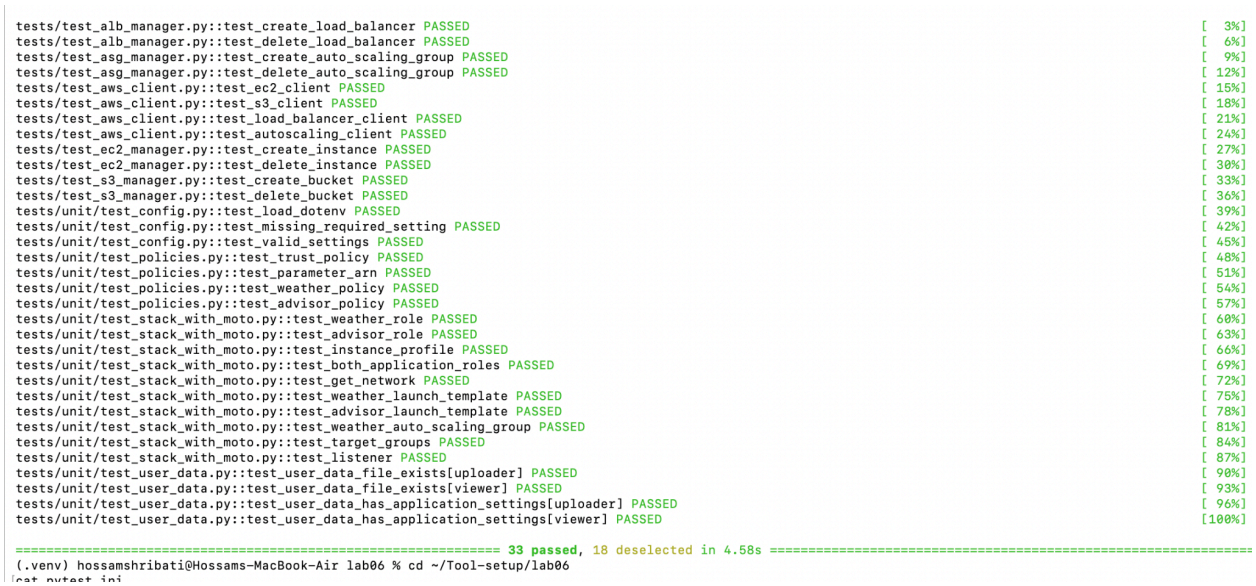
Screenshot 12 — Advisor Chaos Test Passed.



```
------------------------------- 1 passed in 98.69s (0:01:38) --------------------------------
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % python -m pytest 'tests/chaos/test_recovery
.py::test_application_recovery[advisor]' -m chaos -v -s
========================================= test session starts ==========================================
platform darwin -- Python 3.13.1, pytest-9.2.0, pluggy-1.7.0 -- /Users/hossamshribati/Tool-set
up/lab06/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/hossamshribati/Tool-setup/lab06
configfile: pytest.ini
collected 1 item

tests/chaos/test_recovery.py::test_application_recovery[advisor]
advisor: terminating i-090949cf8e0982b64
advisor: recovery time = 98.88769587498973 seconds (SLO = 180 seconds)
PASSED

================================== 1 passed in 99.79s (0:01:39) ==================================
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```



```
tests/test_alb_manager.py::test_create_load_balancer PASSED                                    [  3%]
tests/test_alb_manager.py::test_delete_load_balancer PASSED                                    [  6%]
tests/test_asg_manager.py::test_create_auto_scaling_group PASSED                               [  9%]
tests/test_asg_manager.py::test_delete_auto_scaling_group PASSED                               [ 12%]
tests/test_aws_client.py::test_ec2_client PASSED                                               [ 15%]
tests/test_aws_client.py::test_s3_client PASSED                                                [ 18%]
tests/test_aws_client.py::test_load_balancer_client PASSED                                     [ 21%]
tests/test_aws_client.py::test_autoscaling_client PASSED                                       [ 24%]
tests/test_ec2_manager.py::test_create_instance PASSED                                         [ 27%]
tests/test_ec2_manager.py::test_delete_instance PASSED                                         [ 30%]
tests/test_s3_manager.py::test_create_bucket PASSED                                            [ 33%]
tests/test_s3_manager.py::test_delete_bucket PASSED                                            [ 36%]
tests/unit/test_config.py::test_load_dotenv PASSED                                             [ 39%]
tests/unit/test_config.py::test_missing_required_setting PASSED                                [ 42%]
tests/unit/test_config.py::test_valid_settings PASSED                                          [ 45%]
tests/unit/test_policies.py::test_trust_policy PASSED                                          [ 48%]
tests/unit/test_policies.py::test_parameter_arn PASSED                                         [ 51%]
tests/unit/test_policies.py::test_weather_policy PASSED                                        [ 54%]
tests/unit/test_policies.py::test_advisor_policy PASSED                                        [ 57%]
tests/unit/test_stack_with_moto.py::test_weather_role PASSED                                   [ 60%]
tests/unit/test_stack_with_moto.py::test_advisor_role PASSED                                   [ 63%]
tests/unit/test_stack_with_moto.py::test_instance_profile PASSED                               [ 66%]
tests/unit/test_stack_with_moto.py::test_both_application_roles PASSED                         [ 69%]
tests/unit/test_stack_with_moto.py::test_get_network PASSED                                    [ 72%]
tests/unit/test_stack_with_moto.py::test_weather_launch_template PASSED                        [ 75%]
tests/unit/test_stack_with_moto.py::test_advisor_launch_template PASSED                        [ 78%]
tests/unit/test_stack_with_moto.py::test_weather_auto_scaling_group PASSED                     [ 81%]
tests/unit/test_stack_with_moto.py::test_target_groups PASSED                                  [ 84%]
tests/unit/test_stack_with_moto.py::test_listener PASSED                                       [ 87%]
tests/unit/test_user_data.py::test_user_data_file_exists[uploader] PASSED                      [ 90%]
tests/unit/test_user_data.py::test_user_data_file_exists[viewer] PASSED                        [ 93%]
tests/unit/test_user_data.py::test_user_data_has_application_settings[uploader] PASSED         [ 96%]
tests/unit/test_user_data.py::test_user_data_has_application_settings[viewer] PASSED           [100%]

============================================================ 33 passed, 18 deselected in 4.58s =============================================================
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % cd ~/Tool-setup/lab06
cat pytest.ini
```

Screenshot 13 — Lab 06 Folder Structure

```
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % history 1 | grep 'chaos_test.py'
 1442  grep -n -E 'weather|advisor|recovery_|csv' testing/chaos_test.py
 1443  sed -n '1,85p' testing/chaos_test.py
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % cd ~/Tool-setup
ls -R lab06 | head -40
__pycache__
advisor-app
create_app_stack_backup.sh
create_database.py
create.py
delete_app_stack_backup.sh
delete_database.py
delete.py
documentation
infrastructure
lab06-ssm-policy.json
previous
pytest.ini
requirements.txt
stack
stack_outputs_lab05_backup.env
stack_outputs_lab05_backup.json
stack_outputs.json
testing
tests
trust-policy.json
uploader
uploader-instance-id.txt
uploader-launch-data.json
uploader-policy.json
user-data
viewer
viewer-instance-id.txt
viewer-launch-data.json
viewer-policy.json
weather-app

lab06/__pycache__:
create_database.cpython-313.pyc
create.cpython-313.pyc
delete_database.cpython-313.pyc
delete.cpython-313.pyc

lab06/advisor-app:
app.js
(.venv) hossamshribati@Hossams-MacBook-Air Tool-setup %
```

```
-------------------------------------
|          DescribeInstances          |
+----------------------+--------------+
|  i-013dfe5f03c1ff96d |  terminated  |
|  i-0d2989185c3b467e8 |  terminated  |
+----------------------+--------------+
(.venv) hossamshribati@Hossams-MacBook-Air lab06 % python delete_database.py
Checking Lab 06 resource ownership...
Ownership checks passed.
Starting Lab 06 database cleanup...
Deleting database: itmo-444-544-weather-db
Waiting for database deletion...
Database deleted.
Database subnet group deleted.
Database security group deleted.
Database password parameter deleted.
WARNING: Database snapshots still exist.
Snapshot: itmo-444-544-lab06-final-backup
Database cleanup completed.
Check AWS for remaining snapshots and backups.
(.venv) hossamshribati@Hossams-MacBook-Air lab06 %
```

Weather Service and AI Advisor documentation

1. Introduction

In this lab, I made two applications using AWS and Python. The first application shows the weather when someone enters a ZIP code. The second application uses Google Gemini AI to give advice based on the weather.

I used AWS EC2, Auto Scaling, a Load Balancer, IAM, Systems Manager, and PostgreSQL on Amazon RDS.

I tested both applications to see if they worked. I also tested what would happen if an EC2 instance stopped working.

After I finished the lab, I deleted the AWS resources so I would not get extra charges.

2. AWS Architecture and Setup

I made two applications called Weather Service and AI Advisor.

The Weather Service gets weather information from the National Weather Service API. It also uses a PostgreSQL database to save weather information.

The AI Advisor uses the weather information and Google Gemini AI to give advice to users.

I used these AWS services:

- EC2: Runs my applications.
- Load Balancer: Sends users to the correct application.
- Auto Scaling: Replaces EC2 instances when they stop working.
- Amazon RDS: Saves weather information and search history.
- IAM: Controls what each application can access.
- Systems Manager: Keeps the database password and Gemini API key safe.

I made the database private and turned on encryption to protect the data.

Each application had its own IAM role. This helped control which passwords each application could use.

I used Python and boto3 to create and manage my AWS resources.

3. Testing Results

I used pytest to check my Python code and applications.

Unit Testing

I ran my unit tests using pytest. My latest test showed 33 passed.

I also used Moto to test my AWS code without creating real AWS resources.

This was helpful because I could test my code without spending money on AWS.

Integration Testing

I ran 16 integration tests, and all 16 passed.

These tests checked my AWS resources, such as the Load Balancer, EC2 settings, IAM roles, and database.

I also checked my applications using their web addresses.

Recovery Testing

I tested what happened when an EC2 instance stopped working.

My results were:

- Weather Service: 95.14 seconds to recover.
- AI Advisor: 98.89 seconds to recover.

Auto Scaling replaced the instances, and the applications started working again.

This showed me that Auto Scaling can help keep an application running when something goes wrong.

4. Weather Cache and Database Testing

I used PostgreSQL on Amazon RDS to save weather information and search history.

Weather Cache

I used ZIP code 60616 to check how the weather cache worked.

The first time I searched, the application needed to get new weather information. This is called a cache miss.

When I searched for the same ZIP code again, the application could use information that was already saved. This is called a cache hit.

Using the cache can make the application faster because it does not need to get new weather information every time.

Database Testing

I checked the /history page to see the saved weather searches.

The database was private and encrypted. Security groups controlled which resources could connect to it.

I also checked how the application handled database problems.

The Weather Service was made to keep getting weather information even when the database was not working. However, it could not use the cache during that time.

How the Weather Cache Works

The Weather Service first checks the database to see if weather information is already saved.

If the information is saved and is still new enough, the application uses it. The response shows source: 'cache'.

If the information is not saved or is too old, the application gets new weather information. The response shows source: 'live'.

The cache lasts 600 seconds, which is 10 minutes.

I think the cache is useful because it can save time and reduce the number of API requests.

Database Cleanup

After testing, I deleted the PostgreSQL database and its snapshot.

I also checked AWS to make sure they were removed.

5. Security and Privacy Review

Secret Storage

I used AWS Systems Manager to save the Gemini API key and database password.

This is safer than putting passwords directly in my code.

However, passwords can still be exposed if someone puts them in logs, screenshots, Terminal commands, or GitHub.

IAM Permissions

Each application had its own IAM role.

The Weather Service could read the database password. The AI Advisor could read the Gemini API key.

I think this is important because each application should only access what it needs.

If both applications had full access to Systems Manager, they could read passwords they did not need.

Network Security

The Load Balancer was open to the internet so users could access the applications.

The EC2 instances and database were protected using security groups.

Both applications used the same security group. This was not the safest setup because the AI Advisor could possibly connect to the database.

I think using separate security groups would be better.

Database Security

My application used the database master user.

A better option would be to create another database user with only the permissions the application needs.

The database also used encryption and TLS to protect information.

TLS certificate checking helps make sure the application is connecting to the correct database.

I also used SQL placeholders to help protect against SQL injection.

Backups and Data

I turned off automatic database backups because this was only a temporary lab.

This helped reduce backup storage use, but it also meant I could lose saved information if something happened to the database.

For a real application, I would turn on backups and make a plan to recover lost data.

I would also follow company rules about how long information should be saved.

AI Privacy and Security

The AI Advisor sent ZIP codes and public weather information to Gemini.

It did not need to send personal information.

If I used this application with real users, I would first check Gemini's privacy rules.

The application also escaped AI-generated text before showing it on the website. This helped protect the website from harmful HTML or JavaScript.

Cost and Availability

If many people used the AI Advisor at the same time, they could use up the Gemini API limit.

To help prevent this, I would add request limits and user login.

The application could also show a backup answer when Gemini was not working.

I think users should know when the answer comes from a backup instead of Gemini.

When the database was not working, the Weather Service could still get weather information, but it could not use saved cache information.

6. Experiments and Failure Testing

Weather Service Testing

I tested the Weather Service using ZIP code 60616.

I checked that it could get weather information and show the result in JSON.

I also checked how the application handled incorrect ZIP codes.

IAM Security Testing

I checked the IAM permissions for both applications.

The goal was to make sure each application could access only the secret it needed.

This is important because giving an application too many permissions can create security problems.

Database Failure Testing

I checked how the Weather Service handled a database connection problem.

The application was designed to keep getting weather information even if the database was not available.

However, it could not use the database cache.

This showed me why applications should be able to handle problems with other services.

Gemini AI Failure Testing

I checked how the AI Advisor handled Gemini problems.

The application used a backup answer when Gemini was not available.

I think this is useful because users can still get basic advice.

However, the application should tell users when the answer is a backup answer.

EC2 Recovery Testing

I tested Auto Scaling by terminating EC2 instances.

The Weather Service recovered in 95.14 seconds.

The AI Advisor recovered in 98.89 seconds.

This showed me that Auto Scaling could replace the failed instances.

The database was also separate from the EC2 instances. This is important because saved information should not disappear when an instance is replaced.

7. AWS Cleanup

After I finished testing, I deleted my AWS resources to avoid extra charges.

I deleted the EC2 instances, Auto Scaling Groups, Load Balancer, target groups, launch templates, IAM roles, and PostgreSQL database.

I also deleted the database snapshot and the Systems Manager parameters used for this lab.

I used AWS CLI commands to check that the resources were removed.

The main Lab 06 resources were deleted. I kept the shared security group because I also used it for other labs.

8. What Moto Could Not Test

Moto helped me test my Python AWS code without using real AWS resources.

I used it to test things like IAM roles, security groups, and other AWS settings.

However, Moto cannot test everything that happens in real AWS.

For example, Moto cannot prove that a real PostgreSQL database is working or that its backup settings are correct.

It also cannot prove that security groups are blocking connections in the real network.

Moto cannot fully test TLS connections or make sure the application is checking the real database certificate.

These things need to be checked using real AWS resources.

Moto also cannot fully test how Auto Scaling recovers after an EC2 instance stops working.

That is why I used recovery testing.

My Weather Service recovered in 95.14 seconds, and my AI Advisor recovered in 98.89 seconds.

I think Moto was helpful because it made testing easier and cheaper.

However, I also needed integration and recovery tests to see how my applications worked in AWS.

Why Two API Calls Are Needed

The Weather Service needs two API calls to get weather information.

The first call uses Zippopotam.us to change a ZIP code into latitude and longitude.

The second call uses the National Weather Service API to get the weather forecast.

Both calls are needed because the National Weather Service uses location coordinates instead of ZIP codes.

9. Python vs. Bash Comparison

In this lab, I used Python and boto3 to create and delete AWS resources.

Bash was shorter because I could use simple AWS CLI commands.

Python needed more lines of code, but I think it was easier to organize.

For example, Python made it easier to create functions and handle errors when working with the database.

I could also use pytest and Moto to test my Python code without creating real AWS resources.

Both Python and Bash can manage AWS resources, but I think Python was better for this lab because it made my code easier to test and fix.

10. Security and Privacy Review Questions (Section 15.1)

1. Secret Storage

I saved the Gemini API key and database password in AWS Systems Manager.

However, passwords can still be exposed through Terminal history, logs, screenshots, or GitHub.

I should be careful not to show passwords in commands or upload files that contain them.

2. Least Privilege

Each application had its own IAM role.

If both roles had ssm:* access to everything, they could read or change passwords they did not need.

This would make the system less secure.

3. Network Exposure

The Load Balancer was open to the internet because users needed to access the applications.

The EC2 instances only accepted application traffic from the Load Balancer.

The database only accepted connections from the allowed application security group.

This helped protect the EC2 instances and database.

4. Shared Security Group

Both applications used the same security group.

This means the AI Advisor could possibly connect to the database even though it did not need to.

I would make separate security groups and only allow the Weather Service to connect to the database.

5. Database Login

My application used the database master user.

I think it would be safer to create a different user with fewer permissions.

For example, the user could have only SELECT and INSERT permissions on the tables it needs.

6. Encryption

The database used encryption and TLS.

TLS certificate checking helps make sure the application is connecting to the real database.

Without this check, someone could try to make the application connect to a fake database.

7. SQL Injection

My application used SQL placeholders instead of putting user input directly into SQL commands.

For example:

SELECT data, updated_at FROM weather_cache WHERE zip = $1

This helps protect the database from SQL injection.

Without placeholders, someone might enter harmful text that changes the SQL command.

8. Backups

I turned off automatic backups and deleted the database without keeping a final snapshot.

This was okay for a temporary lab, but it would be risky for a real application.

If the database had important information, I could lose it.

For a real application, I would turn on backups.

9. Data Lifecycle

In a real company, the people responsible for the data should decide how long information needs to be saved.

They should work with the security and legal teams.

They should also follow company rules and laws before deleting information.

10. Untrusted AI Output

The application escaped AI-generated text before showing it on the website.

This is important because AI output might contain harmful HTML or JavaScript.

Without this protection, someone could try to use the AI response to attack the website.

11. Data Sent to a Third Party

The AI Advisor sent ZIP codes and public weather information to Gemini.

If I used a school or company Gemini account, I would first check the privacy rules and get permission if needed.

I would also make sure that private information was not sent to Gemini.

12. Abuse and Cost

Someone could send too many requests to the AI Advisor.

This could use up the Gemini API limit or increase costs.

To help prevent this, I would add request limits, user login, and AWS WAF protection.

13. Availability

If Gemini stopped working, the AI Advisor could show a backup answer.

If the database stopped working, the Weather Service could still get weather information without using the cache.

I think users should know when something is not working normally.

For example, the application could show source: fallback or cache: unavailable.

Conclusion

In this lab, I learned how to build two applications using AWS and Python.

I learned how to use PostgreSQL to save weather information and IAM to control access.

I also learned how to use Systems Manager to keep passwords and API keys safe.

I used pytest and Moto to test my Python code.

I learned that Auto Scaling can help applications recover when EC2 instances stop working.

The biggest thing I learned was that testing is important because it helps me find problems and understand how my applications work.

I also learned that I should delete AWS resources after finishing a lab so I do not get extra charges.